



BIT1123/BISE2093/DIT1113
BIT1123 OBJECT ORIENTED PROGRAMMING

ASSIGNMENT 1 - INDIVIDUAL
(20%)

ALL SECTIONS

FACULTY OF INFORMATION TECHNOLOGY

CITY UNIVERSITY MALAYSIA
CYBERJAYA CAMPUS

PREPARED BY

SIR NAZMIRUL IZZAD BIN NASSIR

Course Name & Code	BIT1123 Object Oriented Programming (Java)
Lecturer Name	Sir Nazmirul Izzad Bin Nassir
Student Name	Hassan Md Rakib
Student ID	202409010095
Program of Study	Bachelor of Software Engineering (Honours)
Assessment Weightage	20% (Individual Assignment)
Submission Date	August 2026

Table of Contents

1. INTRODUCTION	3
2. Knowledge gained in Practical Tutorials.....	3
3. CHALLENGES ENCOUNTERED AND DEBUGGING SOLUTIONS.....	4
4. Knowledge of OOP concepts and understanding of the progression of Java skills.	6
5. FUTURE LEARNING PLANS	6
6. CONCLUSION & REPOSITORY SUBMISSION.....	7

1. INTRODUCTION

In the City University Malaysia Bachelor of Software Engineering (Honours) course, the BIT1123 Object Oriented Programming course is a key component in the program. The primary aim of this module is to move students from simple computational logic to industry standard software design paradigms with the language of Java. The semester included ten extensive lab tutorials, which GA practical experience with OOP, stream-based persistence, even driven GUI, and version control workflow. In this reflective report I will critically examine my learning trajectory, technical achievements, debugging process, and future learning trajectory, as I develop the building blocks of programming skills.

2. Knowledge gained in Practical Tutorials

- Learned the basic syntax of a java program, the use of the standard output stream (`System.out.println`), the use of the Scanner class to get input from the terminal, and multi-way decision making logic (if-else if-else) in `StudentGrade.java` for **week 1**.
- **In Week 2** (Classes, Objects & Constructors) they moved into object orientation, created user-specified class blueprints (`Student.java`), used the new operator to create objects, manipulated object state through user-specified constructors, and bound domain behaviors through object instance methods.
- **In Week 3-4** (Inheritance & Method Overriding), students are working on hierarchical code reuse by using the keyword `extends` to link is-a hierarchies.

Used the super constructor to pass state to the parent classes (Person.java) and runtime polymorphism using the @Override.

- **Data Encapsulation & Access Modifiers (Week 5):** Data is declared as private in Student.java and accessed safely through public getters and setters, ensuring that data is protected from unauthorized external changes, thus promoting data integrity.
- **Week 6:** Protected Scope & Class Extension: Defined domain-specific subclass hierarchies (Employee.java, Lecturer.java) using protected access modifier, allowing direct access of members of the base class in derived classes but not to unrelated classes.
- **In Week 7** (Abstraction & Method Contracts) implemented abstract classes (Appliance.java) with non-negotiable operational contracts (abstract void operate ()) and enforced concrete behavior in subclasses like Washing Machine and Refrigerator.
- **In Week 8-9** (Collections Framework & Stream File I/O), used dynamic array structures (Array List<String>) in manipulating in-memory task list, using strong file serialization and deserialization using Buffered Writer, Buffered Reader and File Reader/File Writer within robust try-catch exception blocks.
- **Week 10** (GUI Development & Event Dispatching): Developed interactive desktop user interface(s) with Java Swing/AWT (J Frame, J Button, J Label) frameworks; implemented interactive user actions by binding to coordinates of GUI components in QuizBattleGUI.java; using the ActionListener interface.

3. CHALLENGES ENCOUNTERED AND DEBUGGING SOLUTIONS

Developing functional software naturally presents structural and syntax obstacles. The table below details key challenges faced during the practical sessions and the technical solutions implemented:

Challenge Category	Specific Problem Encountered	Debugging Strategy & Solution
Constructor Delegation	Subclasses in Week 3–4 failed to compile when calling parent attributes without explicit constructor chaining.	Ensured <code>super(name, id);</code> was strictly declared as the first statement inside the subclass constructor to properly instantiate the parent object.
Encapsulation Scope Errors	Attempting to directly mutate private class variables from <code>Main.java</code> resulted in visibility compilation errors.	Followed strict JavaBean encapsulation standards by calling dedicated setter methods (e.g., <code>setStudentID()</code>) rather than direct field assignments.
Resource Leaks in File I/O	Unclosed file writer streams caused data truncation and locked file handles during Week 8–9 execution.	Implemented the try-with-resources pattern (<code>try (BufferedWriter writer = ...)</code>), ensuring automatic flushing and safe closure of I/O streams.
GUI Component Placement	Swing components overlapped or failed to render when manual absolute coordinates were miscalculated.	Used <code>setLayout(null)</code> combined with precise <code>setBounds(x, y, width, height)</code> dimensions and invoked <code>setLocationRelativeTo(null)</code> for centered presentation.

4. Knowledge of OOP concepts and understanding of the progression of Java skills.

I had a complete “aha!” moment during the sequential tutorials that changed my perception of software engineering. At first, it was as if I was writing fragments of scripts. I found that OOP can help develop a modular, scalable, and secure architecture by making it easier to apply during real practice:

- Encapsulation: Provides clear boundaries, conceals implementation details, preserves object state.
- Inheritance: Avoids boiler plumbing by sharing common properties among special entities.
- Polymorphism: enables programs to call uniform interfaces at runtime but achieve one of several different behaviors.
- Abstraction: Detaches conceptual requirements from the technical implementation so that the system can be implemented without clutter.

5. FUTURE LEARNING PLANS

With a good grasp of core Java and OOP, the next steps I would like to take in my development are:

- To learn advanced Data Structures and Algorithms (Trees, Graphs, Stacks, Queues) and Big-O in Java.
- Using JDBC and Hibernate ORM to connect Java applications with relational database management systems (MySQL/PostgreSQL)
- Deeper understanding of the Spring Boot ecosystem to create scalable enterprise RESTful microservices and web backends.

6. CONCLUSION & REPOSITORY SUBMISSION

I have been able to consolidate my programming skills, discipline in version control (using Git and GitHub) and my thinking in relation to object-oriented software design in the practical lab tutorials in BIT1123. The necessary source code for all the weeks from 1 to 10 is completely integrated and managed within my single GitHub repository and is accompanied by documentation and project assets.

GitHub Repository Deliverable URL:

https://github.com/rakibhassan1224/RAKIB_object-oriented-programming-tutorials.git